

Voice Web Assistant - One Page Report

Goal: accept voice input, fetch live internet data when needed, apply guardrails, and speak the result back through a polished browser interface.

1. Human-like avatar interface

The implemented app includes a lightweight animated AI avatar that reacts while the assistant is processing. For a production photorealistic version, the same response pipeline would connect ElevenLabs or another low-latency TTS stream into a real-time avatar provider such as Tavus, HeyGen Streaming Avatar, D-ID, or NVIDIA ACE. The flow is: user microphone -> STT -> LLM/tool response -> streaming TTS -> avatar lip-sync/video stream -> browser. This keeps the hard video synthesis outside the core app, reduces build risk, and still creates a realistic human-like interface. A production avatar would also support barge-in, emotion tags, mouth-viseme sync, and WebRTC delivery for low delay.

2. Tools and models used

Layer	Choice	Why
Frontend	React + Vite	Fast, clean UI with MediaRecorder audio capture.
Backend	FastAPI	Async Python API, easy file upload and tool orchestration.
STT	OpenAI Whisper API	Reliable speech-to-text with minimal setup.
LLM Agent	Gemini 2.5 Flash	Fast responses, tool calling, strong cost/performance.
Live Data	OpenWeather + Tavily	Weather and general internet lookup through explicit tools.
TTS	ElevenLabs	Natural spoken responses with simple API integration.
Guardrails	Input + output safety checks	Refuse unsafe requests before and after generation.

3. Reducing lag and latency

The MVP already uses a simple async backend. To reduce latency further, the production version would stream each stage instead of waiting for full completion: streaming STT while the user speaks, Gemini token streaming, and streaming TTS so audio starts as soon as the first sentence is ready. Weather/search calls should run asynchronously, API clients should reuse HTTP connections, and common facts like weather can be cached for a short TTL. For avatar video, WebRTC is preferred over generating and downloading full videos. Additional optimizations include warm containers, smaller fast models for routing, parallel tool execution, request timeouts, and fallback responses when a tool fails.

Architecture

Mic -> React MediaRecorder -> FastAPI -> Whisper STT -> Input guardrail -> Gemini tool-calling -> Weather/Search tool -> Output guardrail -> ElevenLabs TTS -> Browser audio/avatar.

Safety and fit-and-finish

The app refuses clearly unsafe or offensive requests, avoids inventing live facts, shows transcript and answer cards, supports typed fallback testing, and continues to return text even when TTS keys are missing. This keeps the demo robust instead of brittle - because demos love breaking five minutes before submission.